

Home / My Interactive Sessions

Interactive Apps

Desktops

Cluster Desktop
Session

Servers

Jupyter Lab

VSCode Server

VSCode Server (1290189)

1 node | 2 cores | Running

Host: bignode2.priv.bmi.emory.edu

Cancel

Created at: 2026-09-16 11:56:21 EDT

Time Remaining: 59 minutes

Session ID: 3da29ca1-0071-417c-952d-27c9ee4168c6

Connect to VS Code

VSCode Server (1289969)

Completed | Edit | Refresh

Created at: 2026-09-16 07:34:25 EDT

Delete

Session ID: d0e161ec-72ae-48b5-93fa-3c805fc3f74e

For debugging purposes, this card will be retained for 6 more days

(1): log onto server

python_quiz

September 16, 2026

1 BMI500 Python Quiz

1.1 Problem 1

```
[1]: def leibniz_sum(n):  
    total = 0.0  
    for k in range(n):  
        total += (-1) ** k / (2 * k + 1)  
    return total  
  
    #Examples for tests  
print("4 * S_10 =", 4 * leibniz_sum(10))  
print("4 * S_10000 =", 4 * leibniz_sum(10000))
```

4 * S_10 = 3.0418396189294032

4 * S_10000 = 3.1414926535900345

1.2 Problem 2

1.2.1 2a.

```
[2]: def leibniz_modulo(n):  
    total = 0.0  
    for k in range(n):  
        term = 1.0 / (2 * k + 1)  
        if k % 2 == 0:  
            total += term  
        else:  
            total -= term  
    return total  
  
print("4 * S_10000 =", 4 * leibniz_modulo(10000))
```

4 * S_10000 = 3.1414926535900345

1.2.2 2b.

```
[3]: def leibniz_power(n):
      total = 0.0
      for k in range(n):
          total += (-1) ** k / (2 * k + 1)
      return total

      print("4 * S_10000 =", 4 * leibniz_power(10000))
```

4 * S_10000 = 3.1414926535900345

1.2.3 2c.

```
[4]: def leibniz_list(n):
      terms = [(-1) ** k / (2 * k + 1) for k in range(n)]
      return sum(terms)

      print("4 * S_10000 =", 4 * leibniz_list(10000))
```

4 * S_10000 = 3.1414926535900345

1.2.4 2d.

```
[5]: def leibniz_set(n):
      terms = {(-1) ** k / (2 * k + 1) for k in range(n)}
      return sum(terms)

      print("4 * S_10000 =", 4 * leibniz_set(10000))
```

4 * S_10000 = 3.1414926535900554

1.2.5 2e.

```
[6]: def leibniz_dict(n):
      terms = {k: (-1) ** k / (2 * k + 1) for k in range(n)}
      return sum(terms.values())

      print("4 * S_10000 =", 4 * leibniz_dict(10000))
```

4 * S_10000 = 3.1414926535900345

1.2.6 2f.

```
[7]: import numpy as np

      def leibniz_numpy(n):
          """Build a NumPy array of the first n terms, then sum it (vectorized)."""
```

```

k = np.arange(n)
terms = (-1.0) ** k / (2 * k + 1)
return float(terms.sum())

print("4 * S_10000 =", 4 * leibniz_numpy(10000))

```

Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.

Intel MKL WARNING: Support of Intel(R) Streaming SIMD Extensions 4.2 (Intel(R) SSE4.2) enabled only processors has been deprecated. Intel oneAPI Math Kernel Library 2025.0 will require Intel(R) Advanced Vector Extensions (Intel(R) AVX) instructions.

4 * S_10000 = 3.1414926535900416

1.2.7 2g.

```

[8]: def leibniz_numpy_indexing(n):
    k = np.arange(n)
    terms = (-1.0) ** k / (2 * k + 1)

    positive = terms[::2]
    negative = terms[1::2]

    return float(positive.sum() + negative.sum())

print("4 * S_10000 =", 4 * leibniz_numpy_indexing(10000))

```

4 * S_10000 = 3.141492653590044

1.2.8 2j.

```

[9]: def leibniz_paired(n):
    total = 0.0
    for m in range(n // 2):
        total += 2.0 / ((4 * m + 1) * (4 * m + 3))
    if n % 2 == 1:
        total += 1.0 / (2 * (n - 1) + 1)
    return total

def leibniz_paired_numpy(n):
    m = np.arange(n // 2)
    total = (2.0 / ((4 * m + 1) * (4 * m + 3))).sum()
    if n % 2 == 1:

```

```

        total += 1.0 / (2 * n - 1)
    return float(total)

for n in (10000, 10001):
    print(f"4 * alternating = {4*leibniz_power(n)!r}")
    print(f"4 * paired loop = {4*leibniz_paired(n)!r}")
    print(f"4 * paired numpy = {4*leibniz_paired_numpy(n)!r}")

```

```

4 * alternating = 3.1414926535900345
4 * paired loop = 3.1414926535900367
4 * paired numpy = 3.1414926535900434
4 * alternating = 3.1416926435905346
4 * paired loop = 3.141692643590537
4 * paired numpy = 3.1416926435905435

```

1.3 Problem 3

```

[10]: import math
import timeit
import pandas as pd

n = 10000
reference = math.pi

implementations = {
    "2a modulo": leibniz_modulo,
    "2b  $(-1)^k$ ": leibniz_power,
    "2c list": leibniz_list,
    "2d set": leibniz_set,
    "2e dict": leibniz_dict,
    "2f numpy": leibniz_numpy,
    "2g numpy slicing": leibniz_numpy_indexing,
    "2j paired numpy": leibniz_paired_numpy,
    "2j paired loop": leibniz_paired,
}

repeats = 3
results = []

for name, func in implementations.items():
    seconds = timeit.timeit(lambda f=func: f(n), number=repeats) / repeats

    approx = 4 * func(n)
    error = abs(approx - reference)

    results.append({
        "Implementation": name,

```

```

        "4 * S_n": approx,
        "Absolute Error": error,
        "Time (ms)": seconds * 1e3,
    })

df = pd.DataFrame(results)

df = df.sort_values("Time (ms)", ascending=True).reset_index(drop=True)
display(
    df.style.format({
        "4 * S_n": "{:.17f}",
        "Absolute Error": "{:.12e}",
        "Time (ms)": "{:.3f}",
    })
)

df = df.sort_values("Absolute Error", ascending=True).reset_index(drop=True)
display(
    df.style.format({
        "4 * S_n": "{:.17f}",
        "Absolute Error": "{:.12e}",
        "Time (ms)": "{:.3f}",
    })
)

```

<pandas.io.formats.style.Styler at 0x116dcfd50>

<pandas.io.formats.style.Styler at 0x116de1510>

In the experiment, we set $n = 10000$, and compare the accuracy/speed/clarity of different methods.

Which is the most accurate?

At this n , all of them are very accurate, with an absolute error less than $1e - 4$. The “2d set” method shows the smallest absolute error. However, the tiny differences between them are negligible compared with the truncation error.

Which is the fastest?

The computation time vary significantly between methods. The fastest one is the “2j paired numpy” method. The reason is that it does not require a for-loop iteration and numpy itself is implemented for efficient computation (e.g., the operations are vectorized).

Which is the clearest?

From my perspective, 2a/2c method is the most clear implementation. Because it directly translates the equation into code.

Which function would I use to calculate π ?

I would use “2j paired numpy” method to calculate π , because this method is fast and accurate. It maintains high accuracy while achieving satisfying efficiency.

1.4 Problem 4

```
[11]: import numpy as np
import math
import matplotlib.pyplot as plt

n_values = np.unique(np.logspace(0, 7, 140).astype(int))

error = np.array([
    abs(4 * leibniz_paired_numpy(int(n)) - math.pi)
    for n in n_values
])

fig, ax = plt.subplots(figsize=(7, 5))

ax.loglog(
    n_values,
    error,
    ".",
    ms=4,
    color="black",
    label="Absolute error"
)

ax.loglog(
    n_values,
    2.0 / n_values,
    "-",
    lw=1.2,
    color="red",
    label=r"Reference slope  $-1$ "
)

ax.set_xlabel(r"Number of terms  $n$ ", fontsize=13)
ax.set_ylabel(r"Absolute error  $|4S_n - \pi|$ ", fontsize=13)
ax.set_title("Leibniz Series: Error vs. Number of Terms", fontsize=14)
ax.tick_params(axis="both", which="major", labelsize=11)
ax.tick_params(axis="both", which="minor", labelsize=9)
ax.legend(
    fontsize=11,
    loc="upper right",
    frameon=False
)

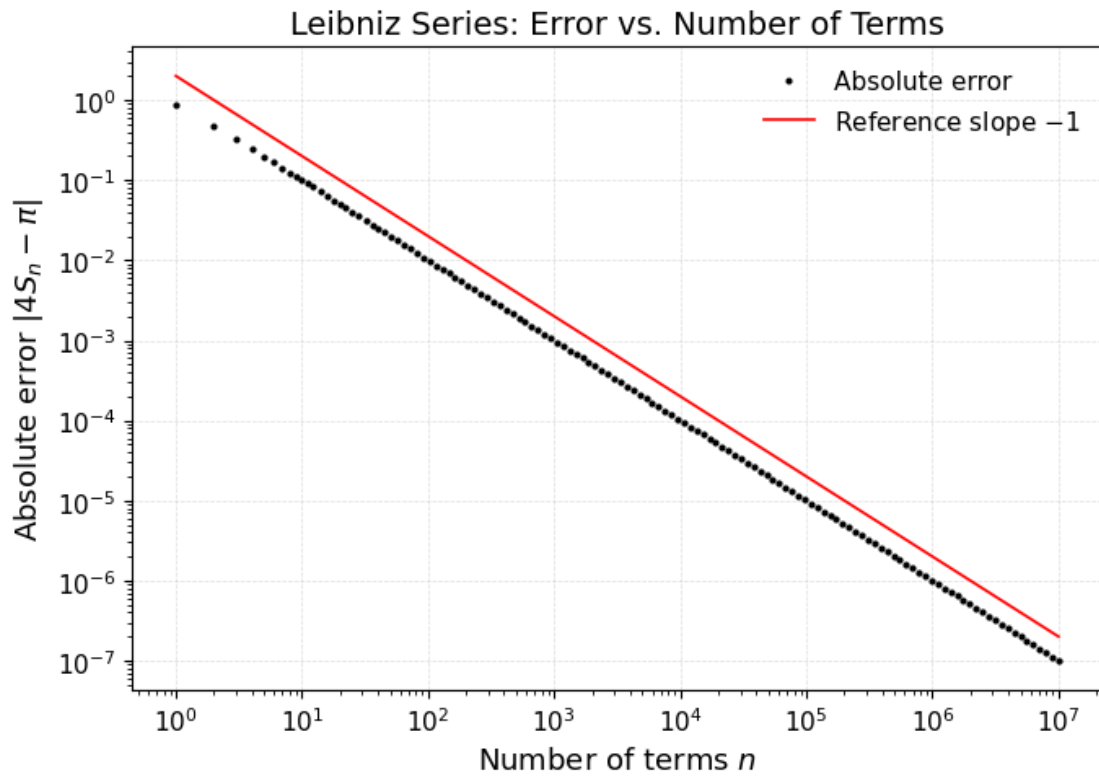
ax.grid(
    True,
    which="major",
```

```

        linestyle="--",
        linewidth=0.5,
        alpha=0.4
    )

fig.tight_layout()
plt.show()

```



I use the paired NumPy implementation (2j), since it is fast and accurate. Both axes are set to be logarithmic. and the figure is made (`figsize=(7, 5)`) rather than (2,2) as suggested in the instruction, because the figure is too small to read. We can clearly see the convergence behavior.

```

Session date ..... 2026-09-13
Assignment ..... BMI500 W3 HW2: matrix-vector multiplication
Working directory ..... /Users/zhanglige/Desktop/Under+PhD Courses/PhD_FirstYear/BMI500/W3
Files generated ..... bmi500_hw2/matvec_multiply.py
                        bmi500_hw2/matvec_tests.py
Test environment ..... conda environment "bmi500" (as requested by the user)
                        (/Users/zhanglige/opt/anaconda3/envs/bmi500)

```

SECTION 1 – LLM / CODING ASSISTANT (AGENT FRAMEWORK) AND MODEL VERSION

Assistant / agent framework:

- Claude Code: Anthropic's official command-line coding agent (CLI).
- Runtime: Claude Agent SDK, running inside the VSCode native extension.
- Platform: macOS (Darwin 24.6.0), shell zsh.

Model:

- Model requested by the user with the local command: `/model opus`
- Session-reported model after selection: `deepseek-v4-pro`
(command output: "Set model to deepseek-v4-pro")
- The session environment states: "You are powered by the model deepseek-v4-pro".

SECTION 2 – USER PROMPTS (verbatim, in order)

Prompt 1:

"Generate the code based on the instructions written in the matvec_multily.py file. Please use the bmi500 environment if you want to do any potential test."

Prompt 2:

"Could you also double-check possible failure modes for your `matvec_multiply` function (e.g., maybe some problems with particular inputs, or something alike), and add logic to guard them."

Prompt 3:

"Can you document my prompts, the LLM and coding assistant (agent framework) used, version of model, conversations. Save everything in AI_code_gen.txt."

SECTION 3 – CONVERSATION RECORD (assistant actions per user turn)

Turn 1 – Initial implementation (in response to Prompt 1):

1. Listed the W3 directory; the file was not at the top level, so searched recursively and located `bmi500_hw2/matvec_multiply.py` and `bmi500_hw2/matvec_tests.py` (both skeletons with instruction comments). Also read `README.md` and `requirements.txt` for context.
2. Implemented `bmi500_hw2/matvec_multiply.py`:
 - `dot_product(vector1, vector2)`: computes the dot product with a for loop, with docstring and comments; raises `ValueError` on length mismatch.
 - `matvec_multiply(matrix, vector)`: computes the matrix-vector product by applying `dot_product` to each row of the matrix; raises `ValueError` for an empty matrix.
 - `main()`: builds a random `1000x1000` matrix and a random vector of length `1000`, computes the product, and verifies one randomly chosen row against an independent computation.
3. Implemented `bmi500_hw2/matvec_tests.py`: 10 unittest tests (5 for `dot_product`, 5 for `matvec_multiply`), covering known values, empty vectors, negative/zero values, identity matrix, single row, and the two error cases.
4. Tested in the `bmi500` conda environment as requested:
 - `"python -m unittest matvec_tests -v" -> Ran 10 tests, OK.`
 - `"python matvec_multiply.py" -> Result length 1000 (expected 1000); Row 540 check: computed 255.474339, expected 255.474339; "Matrix-vector product computed correctly."`

Turn 2 – Robustness hardening (in response to Prompt 2):

1. Audited possible failure modes of `matvec_multiply`:
 - `matrix = None` or another non-list -> cryptic `TypeError` from `len()`
 - flat list used as a matrix -> scalar "rows", cryptic error

- jagged matrix (rows of unequal len) -> failure only mid-computation
 - row length != vector length -> failure only mid-computation
 - non-numeric entries ("a", None) -> TypeError deep in the loop
 - vector not a list -> cryptic TypeError
2. Added guards:
- dot_product: TypeError if either input is not a list; TypeError if any entry is not an int/float (message includes the offending types and index); ValueError on length mismatch (kept).
 - matvec_multiply: validates up front – TypeError for a non-list matrix/vector/row (with row index), ValueError for an empty matrix, ValueError for any row whose length differs from the vector's (jagged matrix; message includes the row index and both lengths).
3. Added 7 new tests covering each guard (17 tests total).
4. Re-tested in the bmi500 conda environment:
- "python -m unittest matvec_tests -v" -> Ran 17 tests, OK.
 - "python matvec_multiply.py" -> Result length 1000 (expected 1000); Row 125 check: computed 240.204720, expected 240.204720; "Matrix-vector product computed correctly."
- Runtime: 0.263 s total (0.24 s user), i.e. the added validation has negligible cost.
5. Documented one deliberate non-guard: float('nan') is a valid float, so it propagates through the computation as NaN instead of raising (the assertion in main() would catch it). Offered to reject NaN explicitly if desired.

Turn 3 – Documentation (in response to Prompt 3):

1. Created this file, bmi500_hw2/AI_code_gen.txt, containing the user's prompts, the assistant/model information, this conversation record, the test results, and the final generated code (Appendix).

SECTION 4 – TEST RESULTS (bmi500 conda environment)

Unit tests (final run, 17 tests):

```
test_empty_vectors (TestDotProduct) ..... ok
test_known_vectors (TestDotProduct) ..... ok
test_length_mismatch_raises (TestDotProduct) .... ok
test_negative_values (TestDotProduct) ..... ok
test_non_list_input_raises (TestDotProduct) ..... ok
test_non_numeric_entries_raise (TestDotProduct) .. ok
test_zero_vector (TestDotProduct) ..... ok
test_empty_matrix_raises (TestMatVecMultiply) ... ok
test_flat_list_matrix_raises (TestMatVecMultiply). ok
test_identity_matrix (TestMatVecMultiply) ..... ok
test_jagged_matrix_raises (TestMatVecMultiply) ... ok
test_known_product (TestMatVecMultiply) ..... ok
test_matrix_not_a_list_raises (TestMatVecMultiply) ok
test_non_numeric_entry_raises (TestMatVecMultiply) ok
test_row_length_mismatch_raises (TestMatVecMultiply) ok
test_single_row (TestMatVecMultiply) ..... ok
test_vector_not_a_list_raises (TestMatVecMultiply) ok
```

Ran 17 tests in 0.000s – OK

main() run (1000x1000 random data):

```
Result length: 1000 (expected 1000)
Row 125 check: computed 240.204720, expected 240.204720
Matrix-vector product computed correctly.
Runtime: 0.263 s total.
```

SECTION 5 – APPENDIX: FINAL GENERATED CODE

```
--- bmi500_hw2/matvec_multiply.py ---

import random

# create a function to compute the dot product of two vectors using a for loop
# add comments for the selected function

def dot_product(vector1, vector2):
    """Compute the dot product of two vectors using a for loop.
```

The dot product is the sum of the element-wise products of the two vectors, i.e. `sum(vector1[i] * vector2[i])` over all indices `i`.

Args:

`vector1` (list): the first vector
`vector2` (list): the second vector

Returns:

float: the dot product of `vector1` and `vector2`

Raises:

`TypeError`: if either argument is not a list, or if any entry is not an int or float

`ValueError`: if the two vectors do not have the same length

"""

guard: both inputs must be lists

if not isinstance(vector1, list) or not isinstance(vector2, list):
 raise TypeError("Both inputs must be lists of numbers.")

guard: a dot product is only defined for vectors of equal length

if len(vector1) != len(vector2):
 raise ValueError("Vectors must have the same length to compute a dot product.")

result = 0.0

accumulate the element-wise products one index at a time

for i in range(len(vector1)):

guard: every entry must be a number before it can be multiplied

if not isinstance(vector1[i], (int, float)) or not isinstance(
 vector2[i], (int, float))

):

raise TypeError(

f"Vector entries must be ints or floats; got "

f"{type(vector1[i]).__name__} and {type(vector2[i]).__name__} "

f"at index {i}."

)

result += vector1[i] * vector2[i]

return result

create a function to compute the matrix-vector product using the `dot_product` function

add comments for the selected function

def matvec_multiply(matrix, vector):

"""Compute the matrix-vector product using the `dot_product` function.

Treats every row of the matrix as a vector and takes its dot product with the input vector; the `i`-th entry of the result is therefore the dot product of row `i` of the matrix with the vector.

Args:

`matrix` (list of list): an `n x m` matrix stored as a list of `n` rows

`vector` (list): a vector of length `m`

Returns:

list: a vector of length `n`, the matrix-vector product

Raises:

`TypeError`: if the matrix or vector is not a list, if any row is not a list, or if any entry is not an int or float (raised by `dot_product`)

`ValueError`: if the matrix is empty, or if any row has a length different from the vector

"""

guard: the matrix must be a list of rows

if not isinstance(matrix, list):

raise TypeError("Matrix must be a list of rows.")

guard: an empty matrix has no rows, so the product is undefined

if len(matrix) == 0:

raise ValueError("Matrix cannot be empty.")

guard: the vector must be a list

if not isinstance(vector, list):

raise TypeError("Vector must be a list.")

guard: every row must be a list, and its length must match the

vector length (a jagged matrix has no well-defined product)

for i, row in enumerate(matrix):

```

        if not isinstance(row, list):
            raise TypeError(
                f"Row {i} is not a list; the matrix must be a list of rows."
            )
        if len(row) != len(vector):
            raise ValueError(
                f"Row {i} has length {len(row)}, but the vector has length "
                f"{len(vector)}; every row must match the vector length."
            )

    # dot the input vector with every row of the matrix
    result = []
    for row in matrix:
        result.append(dot_product(row, vector))
    return result

# create a main function to test the matrix-vector product function using randomly generated data of size 1000x1000
# add comments for the selected function

def main():
    """Test matvec_multiply with randomly generated 1000x1000 data.

    Builds a random 1000 x 1000 matrix and a random vector of length
    1000, computes the matrix-vector product, and checks a randomly
    chosen entry of the result against an independent computation.
    """
    size = 1000

    # random matrix with 1000 rows and 1000 columns
    matrix = [[random.random() for _ in range(size)] for _ in range(size)]

    # random vector of length 1000
    vector = [random.random() for _ in range(size)]

    # compute the matrix-vector product with our function
    result = matvec_multiply(matrix, vector)

    # sanity checks on the shape of the result
    assert len(result) == size, "Result should have one entry per matrix row."
    print(f"Result length: {len(result)} (expected {size})")

    # verify one randomly chosen entry against an independent computation
    row_index = random.randrange(size)
    expected = sum(matrix[row_index][j] * vector[j] for j in range(size))
    print(
        f"Row {row_index} check: computed {result[row_index]:.6f}, "
        f"expected {expected:.6f}"
    )
    assert abs(result[row_index] - expected) < 1e-9, "Computed entry does not match."
    print("Matrix-vector product computed correctly.")

if __name__ == "__main__":
    main()

--- bmi500_hw2/matvec_tests.py -----

# generate tests for matrix-vector-product function in matvec_multiply.py

# generate tests for dot-product function in matvec_multiply.py

import unittest

from matvec_multiply import dot_product, matvec_multiply

class TestDotProduct(unittest.TestCase):
    """Tests for the dot-product function in matvec_multiply.py."""

    def test_known_vectors(self):
        # 1*4 + 2*5 + 3*6 = 32
        self.assertEqual(dot_product([1, 2, 3], [4, 5, 6]), 32)

    def test_empty_vectors(self):
        # the dot product of two empty vectors is 0
        self.assertEqual(dot_product([], []), 0)

```

```

def test_negative_values(self):
    #  $(-1)*4 + 2*(-5) + (-3)*6 = -32$ 
    self.assertEqual(dot_product([-1, 2, -3], [4, -5, 6]), -32)

def test_zero_vector(self):
    # anything dotted with the zero vector is 0
    self.assertEqual(dot_product([1, 2, 3], [0, 0, 0]), 0)

def test_length_mismatch_raises(self):
    # vectors of different lengths are invalid
    with self.assertRaises(ValueError):
        dot_product([1, 2], [1, 2, 3])

def test_non_list_input_raises(self):
    # inputs that are not lists are invalid
    with self.assertRaises(TypeError):
        dot_product([1, 2], None)
    with self.assertRaises(TypeError):
        dot_product("ab", "cd")

def test_non_numeric_entries_raise(self):
    # entries that are not numbers cannot be multiplied
    with self.assertRaises(TypeError):
        dot_product([1, "a"], [2, 3])
    with self.assertRaises(TypeError):
        dot_product([1, 2], [None, 3])

class TestMatVecMultiply(unittest.TestCase):
    """Tests for the matrix-vector-product function in matvec_multiply.py."""

    def test_identity_matrix(self):
        # the identity matrix times a vector returns the vector unchanged
        identity = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
        vector = [5, -2, 7]
        self.assertEqual(matvec_multiply(identity, vector), vector)

    def test_known_product(self):
        # row 0:  $1*5 + 2*6 = 17$ , row 1:  $3*5 + 4*6 = 39$ 
        matrix = [[1, 2], [3, 4]]
        vector = [5, 6]
        self.assertEqual(matvec_multiply(matrix, vector), [17, 39])

    def test_single_row(self):
        # a 1x3 matrix times a length-3 vector gives a length-1 result
        self.assertEqual(matvec_multiply([[1, 2, 3]], [4, 5, 6]), [32])

    def test_empty_matrix_raises(self):
        # an empty matrix has no rows to dot with the vector
        with self.assertRaises(ValueError):
            matvec_multiply([], [1, 2])

    def test_row_length_mismatch_raises(self):
        # a row longer than the vector makes the dot product undefined
        with self.assertRaises(ValueError):
            matvec_multiply([[1, 2, 3]], [1, 2])

    def test_matrix_not_a_list_raises(self):
        # a matrix that is not a list is invalid
        with self.assertRaises(TypeError):
            matvec_multiply(None, [1, 2])

    def test_flat_list_matrix_raises(self):
        # a flat list has scalar "rows", so it is not a valid matrix
        with self.assertRaises(TypeError):
            matvec_multiply([1, 2, 3], [1, 2, 3])

    def test_jagged_matrix_raises(self):
        # rows of different lengths make the product undefined
        with self.assertRaises(ValueError):
            matvec_multiply([[1, 2, 3], [4, 5]], [1, 2, 3])

    def test_non_numeric_entry_raises(self):
        # a non-numeric matrix entry cannot be multiplied
        with self.assertRaises(TypeError):
            matvec_multiply([[1, "a"], [3, 4]], [5, 6])

    def test_vector_not_a_list_raises(self):
        # a vector that is not a list is invalid

```

```
with self.assertRaises(TypeError):  
    matvec_multiply([[1, 2]], "12")
```

```
if __name__ == "__main__":  
    unittest.main()
```

```
=====  
END OF LOG  
=====
```

(4): Code Profiling

Homework 3 — Code Profiling Report

BMI 500 Introduction to Ethical Data Science and Informatics

Fall 2026

Author: Lige Zhang

Affiliation: Department of Computer Science, Emory University

Date: 2026.9.16

Coarse-grain Timing

We document the coarse-grain timing in Table 1. We find that the time, CPU, and memory usage monotonically increase as the dataset gets larger. In particular, the CPU usage is within the range of 93%-100%, this means that The `--cpus-per-task=1` request is correctly sized. Nevertheless, we find that the peak RSS is simply 1.2GB, which is far less than the memory we requested. Therefore, we should reduce the memory call to 4GB for future tests.

Table 1: Coarse-grain timing in seconds.

Datasets	pbmc3k	pbmc6k	pbmc10k
Wall clock time (s)	27.22	36.25	74.30
Percent of CPU (%)	93	98	100
Maximum resident set size (MB)	851.6	931.8	1,293.1

Instrumented Profiling

We document the instrumented profiling results in Table 2. We find that the number in “total running time” is close to the results in coarse-grain timing, which further verifies the reliability of our results. We find that “neighbors, UMAP, rank_gene_groups” are the three operations that take most computation time; the time consumption for “neighbors and rank_gene_groups” is also increasing with respect to the input data size. We will further elaborate on it in later sections.

Table 2: Instrumented profiling results in seconds.

Datasets	pbmc3k	pbmc6k	pbmc10k	share @10K
imports	3.790	2.748	2.459	3.4 %
scanpy settings	7.456	7.268	7.132	9.7 %
argument parsing	0.001	0.001	0.001	0.0 %
I/O	0.158	0.192	0.672	0.9 %

Datasets	pbmc3k	pbmc6k	pbmc10k	share @10K
filtering	0.059	0.083	0.424	0.6 %
normalization	0.058	0.086	0.465	0.6 %
highly variable genes	0.064	0.081	0.457	0.6 %
scale	0.042	0.076	0.172	0.2 %
PCA	0.351	0.572	1.076	1.5 %
neighbors	4.191	4.231	30.539	41.7 %
louvain clustering	0.143	0.386	0.781	1.1 %
UMAP	8.291	16.079	14.182	19.4 %
write h5ad	0.156	0.210	0.552	0.8 %
rank_genes_groups	1.758	3.539	14.325	19.6 %
TOTAL (instrumented)	26.516	35.551	73.237	100 %

Fine-grain Profiling

We visualize the c-profiling results in Figure1, Figure 2 and Figure 3 for each dataset. We find that within the rank-gene-groups function, the most time-consumption step is the rank computation. In specific, Scanpy's `_ranks()` calls `pandas.core.algorithms.rank`, which dominates the profile in every dataset.

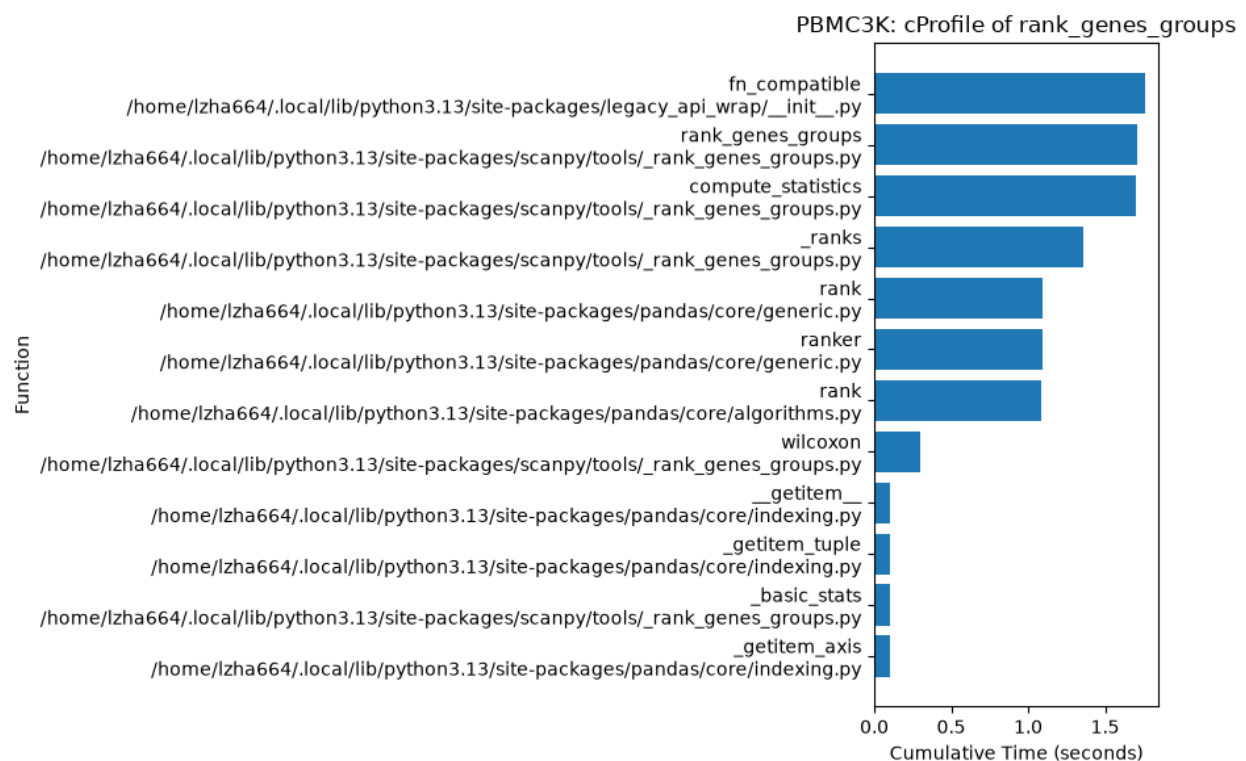


Figure 1: C-Profile for PBMC3K for rank-gene-group.

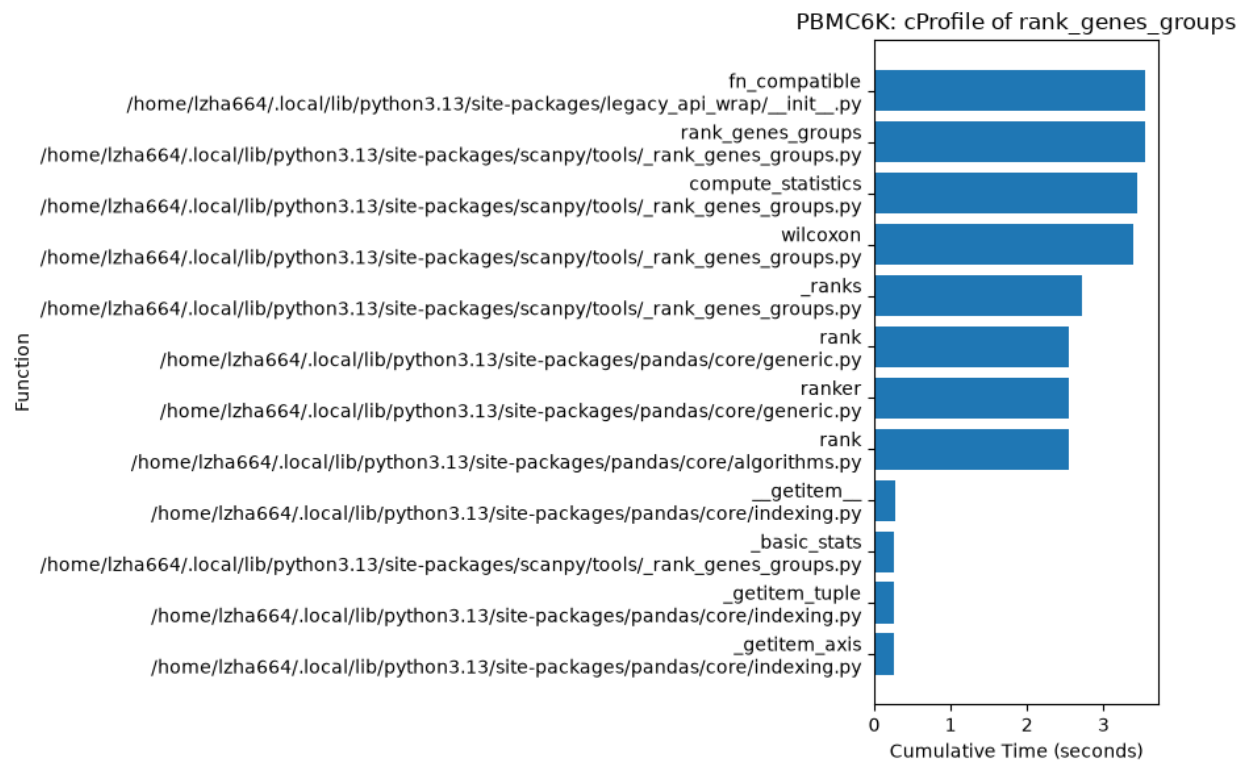


Figure 2: C-Profile for PBMC6K for rank-gene-group.

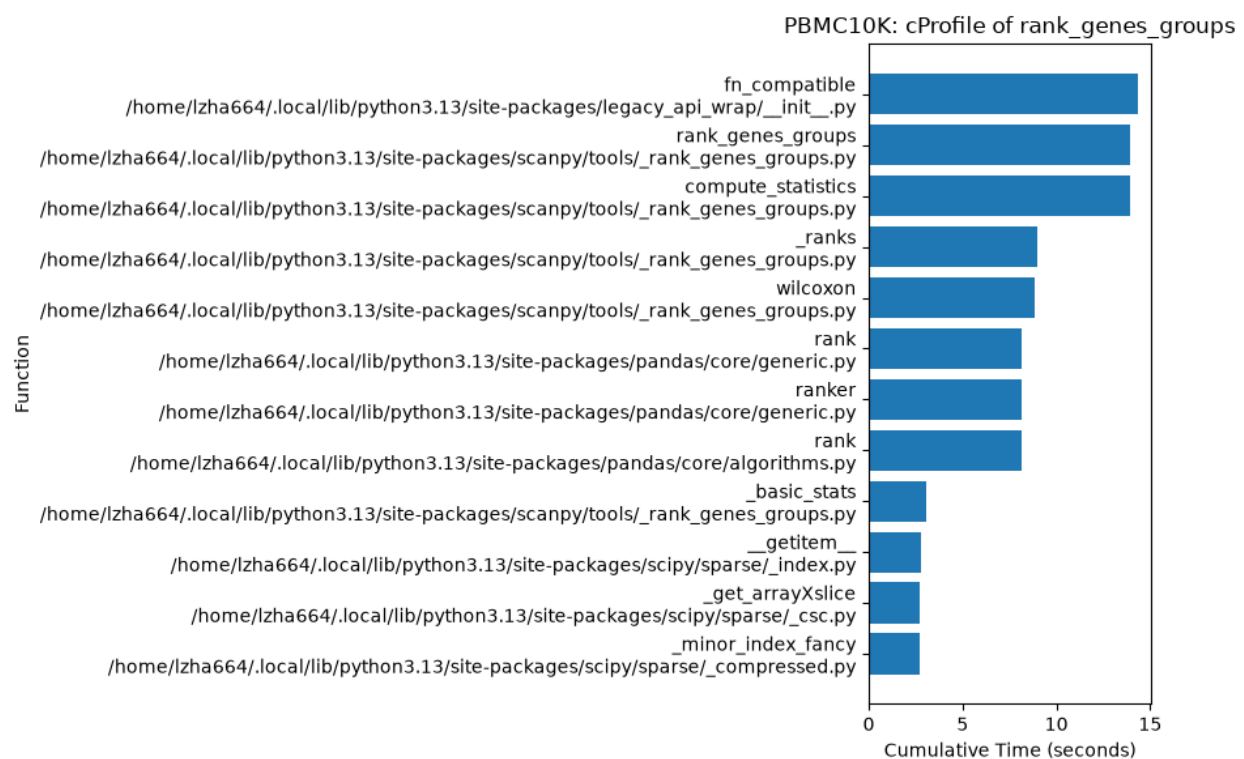


Figure 3: C-Profile for PBMC10K for rank-gene-group.

Runtime growth vs. dataset size

We visualize the time consumption with respect to different steps in Figure 4 and Figure 5. We find that some code sections (in particular, “neighbors, UMAP, rank_gene_groups”) computing time increases as datasets get enlarged, while other code sections do not introduce significant computation overhead.

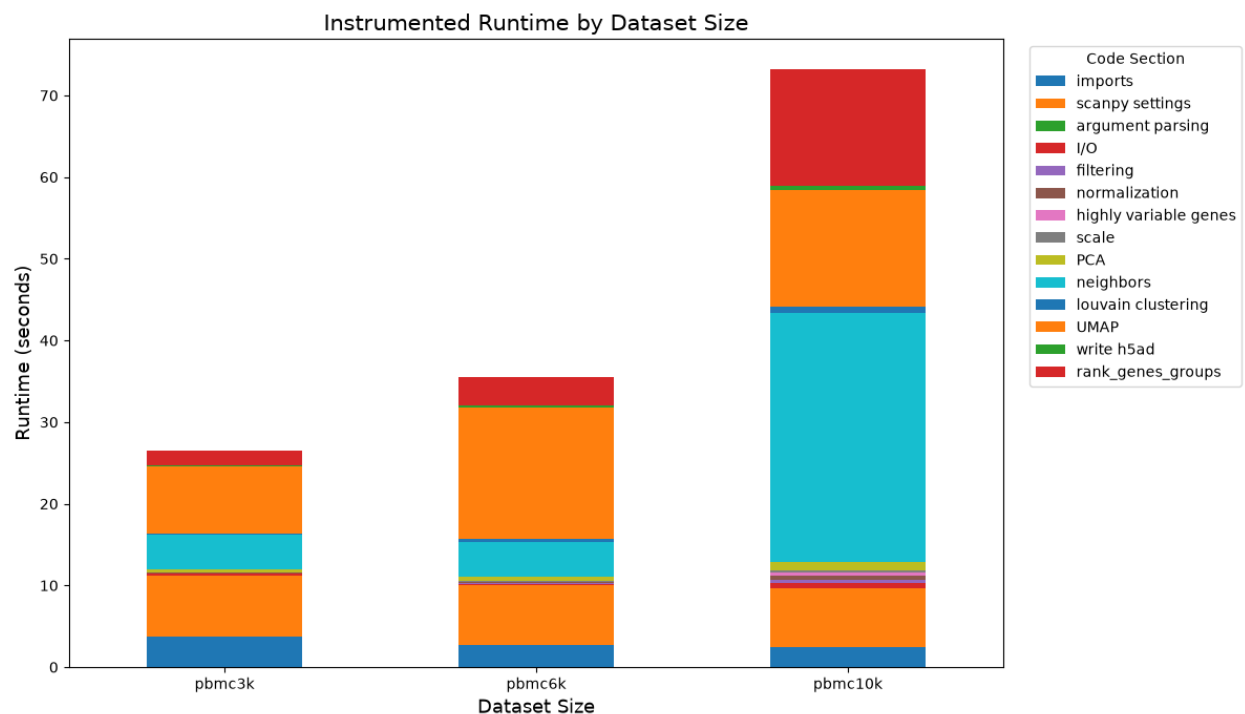


Figure 4: Stacked bar plot for computation time.

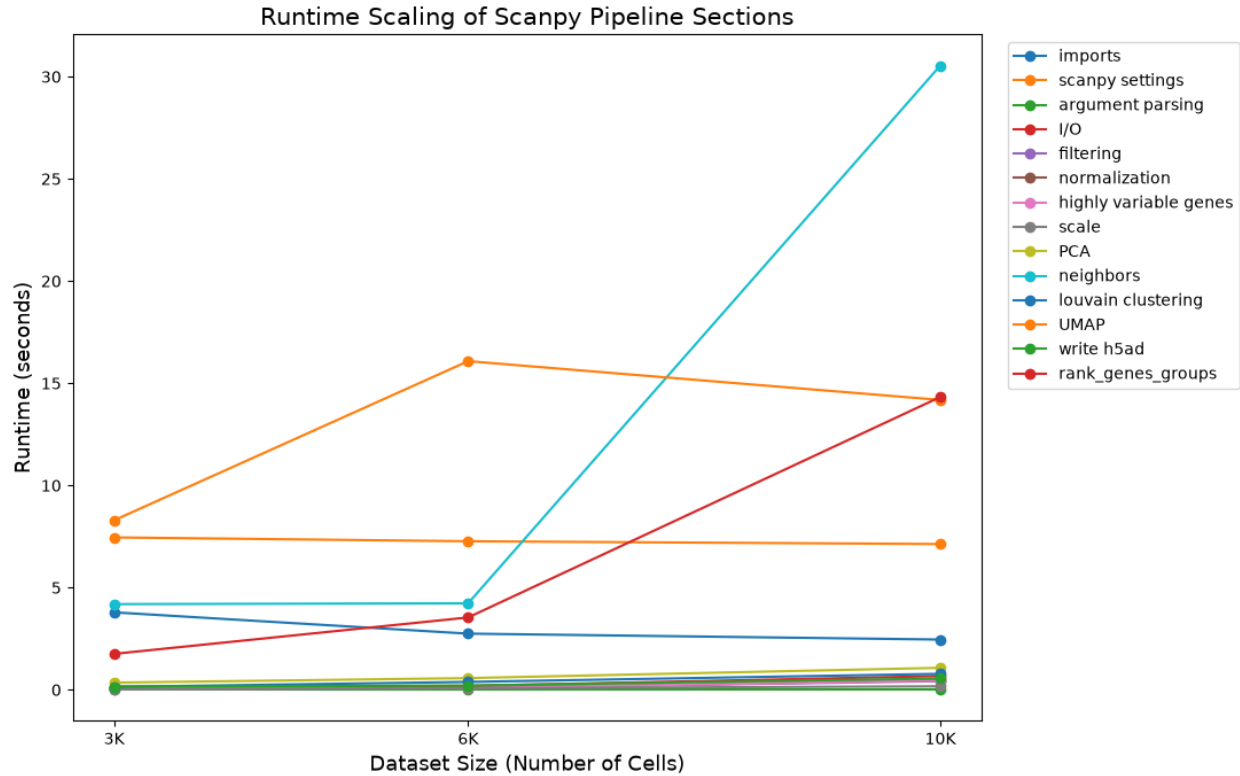


Figure 5: Line plot for computation time.

Bottleneck Analysis and Run-Time Prediction

We identified 2 bottlenecks in total. Specifically, we find that the “neighbors” is the dominant bottleneck at scale; “rank_genes_groups” is the fastest-growing section. In addition, “Umap” is the dominant bottleneck at small/medium scale, nevertheless, it does not seem to be the bottleneck when data size scales up.

Based on our observation, we conduct a least-squares linear regression of each section’s runtime on cell count, fitted over the three measured datasets and extrapolated to 20,000 cells. The result is shown in Table 3 below, and we estimate a run time of 138 seconds.

Table 3: Predicted runtime at PBMC20k by linear regression.

Section	Measured @10K (s)	Predicted @20K (s)	Share of prediction
neighbors	30.539	66.51	48.1 %
rank_genes_groups	14.325	31.74	22.9 %
UMAP	14.182	23.38	16.9 %
scanpy settings	7.132	6.66	4.8 %
PCA	1.076	2.10	1.5 %

Section	Measured @10K (s)	Predicted @20K (s)	Share of prediction
louvain clustering	0.781	1.69	1.2 %
I/O	0.672	1.38	1.0 %
write h5ad	0.552	1.10	0.8 %
normalization	0.465	1.02	0.7 %
highly variable genes	0.457	1.00	0.7 %
filtering	0.424	0.93	0.7 %
imports	2.459	0.49	0.4 %
scale	0.172	0.35	0.3 %
argument parsing	0.001	0.001	0.0 %
TOTAL	73.237	≈ 138.3	100 %

Appendix A. SLURM batch scripts (a, b)

Three identical scripts, differing only in `--job-name` and `--data-set`. Each requests 1 CPU, 16 GB and 5 minutes, and wraps the run in `/usr/bin/time -v` for coarse-grain metrics.

```
#!/bin/bash
#SBATCH --job-name=pbmc3k
#SBATCH --output=pbmc3k_%j.out
#SBATCH --error=pbmc3k_%j.err
#SBATCH --cpus-per-task=1
#SBATCH --mem=16G
#SBATCH --time=0:05:00

set -eo pipefail

cd "$SLURM_SUBMIT_DIR"

source ~/.bashrc
conda activate bmi500

echo "==== Job information ====="
echo "Dataset: pbmc3k"
echo "Node: $(hostname)"
echo "Start time: $(date)"
echo "Working directory: $(pwd)"
echo "===== "

/usr/bin/time -v python scanpy_pbmc.py \
    --data-dir data \
    --data-set pbmc3k \
```

```
--num-threads 1
```

```
echo "End time: $(date)"
```

submit_pbmc6k.sb and submit_pbmc10k.sb are identical, with pbmc3k replaced by pbmc6k and pbmc10k respectively, in the --job-name, --output, --error, echo "Dataset: ..." and --data-set fields.

```
#!/bin/bash
#SBATCH --job-name=pbmc6k
#SBATCH --output=pbmc6k_%j.out
#SBATCH --error=pbmc6k_%j.err
#SBATCH --cpus-per-task=1
#SBATCH --mem=16G
#SBATCH --time=0:05:00
```

```
set -eo pipefail
```

```
cd "$SLURM_SUBMIT_DIR"
```

```
source ~/.bashrc
conda activate bmi500
```

```
echo "==== Job information ====="
echo "Dataset: pbmc3k"
echo "Node: $(hostname)"
echo "Start time: $(date)"
echo "Working directory: $(pwd)"
echo "=====
```

```
/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc6k \
  --num-threads 1
```

```
echo "End time: $(date)"
```

```
#!/bin/bash
#SBATCH --job-name=pbmc10k
#SBATCH --output=pbmc10k_%j.out
#SBATCH --error=pbmc10k_%j.err
#SBATCH --cpus-per-task=1
#SBATCH --mem=16G
#SBATCH --time=0:05:00
```

```
set -eo pipefail

cd "$SLURM_SUBMIT_DIR"

source ~/.bashrc
conda activate bmi500

echo "==== Job information ====="
echo "Dataset: pbmc10k"
echo "Node: $(hostname)"
echo "Start time: $(date)"
echo "Working directory: $(pwd)"
echo "===== "

/usr/bin/time -v python scanpy_pbmc.py \
    --data-dir data \
    --data-set pbmc10k \
    --num-threads 1

echo "End time: $(date)"
```

Appendix B. Console output (c)

pbmc3k (job 1290038)

stdout — pbmc3k_1290038.out

```
==== Job information =====
Dataset: pbmc3k
Node: bignode2.priv.bmi.emory.edu
Start time: Wed Sep 16 08:30:51 EDT 2026
Working directory: /home/lzha664/bmi500_hw2
=====
[timing] imports: 3.790 s
scanpy==1.10.2 anndata==0.12.19 umap==0.5.12 numpy==1.26.4 scipy==1.17.1 pand
as==2.3.3 scikit-learn==1.9.1 statsmodels==0.15.0 igraph==0.11.6 louvain==0.
8.2 pynndescent==0.6.0
using 1 threads
[timing] scanpy settings: 7.456 s
[timing] argument parsing: 0.001 s
[timing] I/O: 0.158 s
[timing] filtering: 0.059 s
[timing] normalization: 0.058 s
[timing] highly variable genes: 0.064 s
[timing] scale: 0.042 s
[timing] PCA: 0.351 s
[timing] neighbors: 4.191 s
```

```
[timing] louvain clustering: 0.143 s
[timing] UMAP: 8.291 s
[timing] write h5ad: 0.156 s
[timing] rank_genes_groups: 1.758 s
[cProfile] saved to pbmc3k_rank_genes_groups.prof
```

```
=== elapsed time per section ===
 3.790 s imports
 7.456 s scanpy settings
 0.001 s argument parsing
 0.158 s I/O
 0.059 s filtering
 0.058 s normalization
 0.064 s highly variable genes
 0.042 s scale
 0.351 s PCA
 4.191 s neighbors
 0.143 s louvain clustering
 8.291 s UMAP
 0.156 s write h5ad
 1.758 s rank_genes_groups
26.516 s TOTAL (instrumented sections)
End time: Wed Sep 16 08:31:18 EDT 2026
```

Resource usage — pbmc3k_1290038.err (/usr/bin/time -v)

```
Command being timed: "python scanpy_pbmc.py --data-dir data --data-set pb
mc3k --num-threads 1"
User time (seconds): 24.32
System time (seconds): 1.17
Percent of CPU this job got: 93%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:27.22
Maximum resident set size (kbytes): 871992
Major (requiring I/O) page faults: 2456
Minor (reclaiming a frame) page faults: 162716
Exit status: 0
```

pbmc6k (job 1290039)

stdout — pbmc6k_1290039.out

```
===== Job information =====
Dataset: pbmc6k
Node: bignode2.priv.bmi.emory.edu
Start time: Wed Sep 16 08:30:59 EDT 2026
Working directory: /home/lzha664/bmi500_hw2
=====
[timing] imports: 2.748 s
scanpy==1.10.2 anndata==0.12.19 umap==0.5.12 numpy==1.26.4 scipy==1.17.1 pand
as==2.3.3 scikit-learn==1.9.1 statsmodels==0.15.0 igraph==0.11.6 louvain==0.
8.2 pyndescent==0.6.0
```

```
using 1 threads
[timing] scanpy settings: 7.268 s
[timing] argument parsing: 0.001 s
[timing] I/O: 0.192 s
[timing] filtering: 0.083 s
[timing] normalization: 0.086 s
[timing] highly variable genes: 0.081 s
[timing] scale: 0.076 s
[timing] PCA: 0.572 s
[timing] neighbors: 4.231 s
[timing] louvain clustering: 0.386 s
[timing] UMAP: 16.079 s
[timing] write h5ad: 0.210 s
[timing] rank_genes_groups: 3.539 s
[cProfile] saved to pbmc6k_rank_genes_groups.prof
```

```
=== elapsed time per section ===
 2.748 s imports
 7.268 s scanpy settings
 0.001 s argument parsing
 0.192 s I/O
 0.083 s filtering
 0.086 s normalization
 0.081 s highly variable genes
 0.076 s scale
 0.572 s PCA
 4.231 s neighbors
 0.386 s louvain clustering
16.079 s UMAP
 0.210 s write h5ad
 3.539 s rank_genes_groups
35.551 s TOTAL (instrumented sections)
End time: Wed Sep 16 08:31:35 EDT 2026
```

Resource usage — pbmc6k_1290039.err (/usr/bin/time -v)

```
Command being timed: "python scanpy_pbmc.py --data-dir data --data-set pb
mc6k --num-threads 1"
User time (seconds): 34.64
System time (seconds): 1.23
Percent of CPU this job got: 98%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:36.25
Maximum resident set size (kbytes): 954116
Major (requiring I/O) page faults: 13
Minor (reclaiming a frame) page faults: 203463
Exit status: 0
```

pbmc10k (job 1290040)

stdout — pbmc10k_1290040.out

```
===== Job information =====
Dataset: pbmc10k
Node: bignode2.priv.bmi.emory.edu
Start time: Wed Sep 16 08:31:03 EDT 2026
Working directory: /home/lzha664/bmi500_hw2
=====
[timing] imports: 2.459 s
scanpy==1.10.2 anndata==0.12.19 umap==0.5.12 numpy==1.26.4 scipy==1.17.1 pand
as==2.3.3 scikit-learn==1.9.1 statsmodels==0.15.0 igraph==0.11.6 louvain==0.
8.2 pynndescent==0.6.0
using 1 threads
[timing] scanpy settings: 7.132 s
[timing] argument parsing: 0.001 s
[timing] I/O: 0.672 s
[timing] filtering: 0.424 s
[timing] normalization: 0.465 s
[timing] highly variable genes: 0.457 s
[timing] scale: 0.172 s
[timing] PCA: 1.076 s
[timing] neighbors: 30.539 s
[timing] louvain clustering: 0.781 s
[timing] UMAP: 14.182 s
[timing] write h5ad: 0.552 s
[timing] rank_genes_groups: 14.325 s
[cProfile] saved to pbmc10k_rank_genes_groups.prof

=== elapsed time per section ===
    2.459 s imports
    7.132 s scanpy settings
    0.001 s argument parsing
    0.672 s I/O
    0.424 s filtering
    0.465 s normalization
    0.457 s highly variable genes
    0.172 s scale
    1.076 s PCA
   30.539 s neighbors
    0.781 s louvain clustering
   14.182 s UMAP
    0.552 s write h5ad
   14.325 s rank_genes_groups
   73.237 s TOTAL (instrumented sections)
End time: Wed Sep 16 08:32:17 EDT 2026
```

Resource usage — pbmc10k_1290040.err (/usr/bin/time -v)

```
Command being timed: "python scanpy_pbmc.py --data-dir data --data-set pb
mc10k --num-threads 1"
User time (seconds): 70.21
System time (seconds): 4.14
```

Percent of CPU this job got: 100%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:14.30
Maximum resident set size (kbytes): 1324180
Major (requiring I/O) page faults: 9
Minor (reclaiming a frame) page faults: 715339
Exit status: 0

Before reviewing, we try to run the provided code test:

```
python matvec_tests.py
```

```
.....
```

```
-----
```

```
Ran 17 tests in 0.000s
```

```
OK
```

The code pass through all the tests, let's start reviewing.

```
#####
```

1. Code well-designed?

Yes, the implementation is correct, the docstrings for each function are clear to users, and `matvec_multiply` actually reuses `dot_product` as the file asks. However, I find two weak points in the design. First, the validation is repeated: `dot_product` checks the type of every entry again on every call, while `matvec_multiply` has already checked the whole row, so an $n * m$ product does $n*m$ extra type checks. Second, the empty input is not handled consistently: `matvec_multiply([], [])` raises a `ValueError`, but `matvec_multiply([[]], [])` silently returns `[0.0]`, even though neither case has anything to multiply.

2. Functionality good for users?

No, the input must be a python list datatype, and it returns an error if this requirement is not satisfied. Nevertheless, in real-world scenarios, we should design functions that can take in different (commonly-used) datatypes, for example a tuple or a numpy array. \

3. UI changes look good?

N/A, this code has no UI. If we treat the function signature as the interface, then I think it looks simple to interact with.

4. Any parallel programming done safely?

N/A, this code does not incorporate any parallel computing.

5. The code isn't more complex than it needs to be?

Yes, it simply uses a for loop, and the logic is simple and clear. The repeated type check I mentioned in comment 1 is the only work that I would call unnecessary, and it does not make the code harder to read.

6. The developer isn't implementing things they might need in the future but don't know they need now?

Yes, but this is in tension with my comment 2. The code only does what the task asks, and I think keeping it this small is the right choice here; supporting other datatypes is my wish for future reuse, maybe it's not something this code needs now.

7. Code has appropriate unit tests?

Yes, `matvec_tests.py` covers the normal cases (e.g., known product, identity matrix) and many error cases (e.g., non-list input, non-numeric entry), so it looks very comprehensive to me.

8. Tests are well-designed.

Yes, I think the assertions are correct, and each test checks one behaviour. But I find two gaps: there is no test with float inputs, and the non-numeric entry is only tested on the matrix side, never on the vector that is passed through `matvec_multiply`.

9. The developer used clear names for everything?

Yes, but I think names could be simplified in certain places (e.g., `vector1` -> `vec1`). Because things will look complicated if we build more functions later on.

10. Comments are clear and useful, and mostly explain why instead of what.

Partly. The comments that explain a reason are useful, for example the ones that say a jagged matrix has no well-defined product, and the error messages that report the row index and both lengths. But other comments only repeat the line below them, like this section in `main()`:

```
# random vector of length 1000
vector = [random.random() for _ in range(size)]
```

We don't need to comment here, it is very easy to identify the code.

11. Code is appropriately documented?

Mostly yes, currently the programmer is only introducing the codebase in the beginning of the file, which is sufficient for now, because it has limited functions. But I would recommend to update the `README.md` to further clarify the function of each script.

12. The code conforms to our style guides?

Mostly yes.

Overall, I think the code is correct and easy to read. The two things I would change are: check the input once instead of on every row, and add a few tests for float inputs and for the vector side.

AI CODE REVIEW RECORD

Date: 2026-09-16

LLM used: GPT-5.6 Sol (OpenAI ChatGPT)

FILES REVIEWED

- 1. matvec_multiply.py
- 2. matvec_tests.py

USER PROMPT

Please perform a careful code review for the files I sent to you. You may refer the following:
<https://smarbear.com/learn/code-review/best-practices-for-peer-code-review/>
<https://google.github.io/eng-practices/review/reviewer/>
<https://www.perforce.com/blog/qac/9-best-practices-for-code-review>
In the end, document the LLM used, my prompts, and the outputs. Save in a ai_code_review.txt and give it back to me.

REFERENCE GUIDANCE USED

The review was informed by the three references supplied by the user.

1. SmartBear, "Best Practices for Code Review"
<https://smarbear.com/learn/code-review/best-practices-for-peer-code-review/>
Relevant principles used here:
 - Keep reviews small and focused.
 - Use a checklist.
 - Record defects clearly.
 - Keep feedback constructive.
 - Use lightweight/tool-assisted review where useful.
2. Google Engineering Practices, "How to do a code review" and "What to look for in a code review"
<https://google.github.io/eng-practices/review/reviewer/>
Relevant principles used here:
 - Review design, functionality, complexity, tests, naming, comments, style, consistency, and documentation.
 - Think through edge cases even when tests pass.
 - Ensure tests are sensible and would fail for broken behavior.
 - Comments should generally explain why, while documentation explains purpose and behavior.
 - Review every human-written line in context.
 - Label review severity so required changes and optional suggestions are distinguishable.
3. Perforce, "Peer Code Review Best Practices"
<https://www.perforce.com/blog/qac/9-best-practices-for-code-review>
Relevant principles used here:
 - Build/test before manual review.
 - Check functionality, performance, test coverage, design, readability, and maintainability.
 - Automate mechanical checks where possible.
 - Make feedback actionable and constructive.

VERIFICATION PERFORMED

Commands/checks performed:

- python -m unittest -v matvec_tests.py
- python -m py_compile matvec_multiply.py matvec_tests.py
- coverage.py statement and branch coverage
- Targeted behavioral probes for:
 - * boolean values
 - * floating-point values
 - * nonnumeric vector entries
 - * zero-column matrix behavior
- Standalone execution of matvec_multiply.py

Observed results:

- 17/17 supplied unit tests passed.
- Both Python files compiled successfully.
- No syntax/runtime error was found in the intended normal-use cases.
- coverage.py for matvec_multiply.py reported 71% statement coverage and 80% branch coverage when the standalone main() demo was included. The missing executable lines/branch were in main() / the __main__ path; dot_product() and matvec_multiply() themselves had all of their executable statements and branches exercised by the supplied tests.
- A standalone 1000x1000 smoke run completed successfully in the review environment. One observed run took about 0.84 seconds and used about

132,264 KB maximum resident memory. This is environment-specific and should not be treated as a portable benchmark.

- `dot_product([True, 2], [3, 4])` returned 11.0 because Python `bool` is a subclass of `int`.

OVERALL REVIEW

Status: Functionally correct; approve with minor/non-blocking changes.

The implementation is small, readable, and correctly computes ordinary matrix-vector products under its documented list-based API. The validation logic gives useful exceptions for malformed dimensions and most invalid types. The tests cover the major normal and error paths well. I did not find a blocking correctness defect for the assignment's apparent intended use.

The strongest parts are:

- Clear separation between `dot_product()` and `matvec_multiply()`.
- `matvec_multiply()` actually reuses `dot_product()`, as requested.
- Helpful docstrings describe arguments, returns, and exceptions.
- Dimension mismatches are rejected before producing a partial result.
- Error messages include useful context such as the failing row/index.
- The test suite includes known outputs, identity behavior, zero/negative cases, malformed matrix shapes, non-list inputs, and nonnumeric matrix values.
- The `__main__` guard prevents the 1000x1000 demo from executing on import.

DETAILED REVIEW COMMENTS

[LOW / CONTRACT] `matvec_multiply.py` lines 37-39:
Boolean values are accepted as numeric entries.

The code checks:

```
isinstance(value, (int, float))
```

In Python, `bool` is a subclass of `int`, so `True` and `False` pass this test.

For example:

```
dot_product([True, 2], [3, 4]) == 11.0
```

The docstring says entries must be "int or float." If boolean values are meant to be invalid, explicitly reject `bool`, for example conceptually:

```
isinstance(x, (int, float)) and not isinstance(x, bool)
```

If booleans are acceptable for this assignment, no change is necessary, but the behavior should be intentional.

[LOW / TEST RELIABILITY] `matvec_multiply.py` lines 117-140:

The standalone demo is randomized and verifies only one randomly selected row.

This is fine as a smoke test, but it is not deterministic, so a failure could be harder to reproduce. Consider using a local seeded generator such as `random.Random(0)`, or using fixed indices for verification.

Also, the correctness checks use Python `assert` statements. Assertions can be disabled with:

```
python -O matvec_multiply.py
```

If `main()` is intended only as an informal demonstration, this is acceptable. If it is intended as a reliable validation program, use explicit checks that raise an exception or rely on the `unittest` suite instead.

[LOW / TEST COMPLETENESS] `matvec_tests.py`:

The tests are good, but several useful behavioral cases are absent even though statement/branch coverage of the reusable functions is already strong.

Recommended additions:

1. Floating-point inputs, using `assertAlmostEqual()`.
2. A rectangular matrix such as 2x3 multiplied by a length-3 vector.
3. A nonnumeric VECTOR entry passed through `matvec_multiply()`, not only a nonnumeric matrix entry.
4. A boolean-entry test if `bool` should be rejected.
5. An explicit test documenting the desired behavior for a zero-length vector with a matrix containing empty rows, e.g. `[[], []]` times `[]`.

This is a good example of why coverage percentage alone is not sufficient: all implementation branches can execute while important API semantics remain unspecified.

[LOW / PERFORMANCE, NON-BLOCKING] `matvec_multiply.py` lines 101-102:

`dot_product()` validates every element of the input vector on every matrix row.

For an $n \times m$ matrix, multiplication already costs $O(n*m)$, so asymptotic complexity does not change. However, the vector's type checks are repeated n times and can add constant-factor overhead for large matrices.

For this educational implementation, clarity and reuse of `dot_product()` are more important, so I would not change it unless performance is a requirement. If this becomes production numerical code, validation could be separated from an internal unchecked dot-product loop, or a numerical library could be used.

[NIT / COMMENTS] `matvec_multiply.py` lines 3-4, 49-50, 106-107 and `matvec_tests.py` lines 1-3:

Some comments appear to be assignment/prompt remnants, for example:

```
# create a function ...  
# add comments for the selected function
```

These do not explain the code's reasoning and can be removed in a polished submission unless the assignment explicitly asks to preserve them.

Many inline comments such as "guard: both inputs must be lists" restate the following line. The assignment appears to request comments, so this is not a problem here, but in production code comments are most valuable when they explain why a decision exists rather than paraphrasing obvious syntax.

[NIT / STYLE] `matvec_multiply.py` line 106:

The comment is longer than a common 88-character formatting target. This is purely stylistic and does not affect correctness.

[NIT / API DESIGN] Both implementation functions:

The API accepts only list objects, rejecting tuples and other sequence types. That behavior matches the current docstrings, so it is not a bug. If the goal is a more general reusable API, accepting `Sequence[Real]` (with suitable runtime validation) would make the functions more flexible. For a beginner assignment, keeping the list-only contract is simpler and reasonable.

[NIT / TYPE CLARITY] `matvec_multiply.py`:

Optional Python type hints would make the public interface easier for editors, static analyzers, and future readers to understand. This is a maintainability improvement, not a correctness requirement.

TEST-SUITE REVIEW

Good coverage:

- Known dot product
- Empty dot product
- Negative values
- Zero vector
- Length mismatch
- Invalid top-level types
- Nonnumeric dot-product entries
- Identity matrix
- Known matrix-vector result
- Single-row matrix
- Empty matrix
- Row/vector dimension mismatch
- Flat-list invalid matrix
- Jagged matrix
- Nonnumeric matrix entry
- Invalid vector top-level type

Suggested additional tests:

- `dot_product([0.1, 0.2], [0.3, 0.4])` with `assertAlmostEqual`
- `matvec_multiply([[1,2,3],[4,5,6]], [7,8,9]) == [50,122]`
- `matvec_multiply([[1,2],[3,4]], [5,"x"])` raises `TypeError`
- bool behavior, according to the intended contract
- zero-column matrix behavior, according to the intended contract

PRIORITY SUMMARY

Blocking issues: None found.

Recommended before submission:

- Decide whether bool should count as a valid numeric input.
- Add one floating-point test and one vector-side nonnumeric test.
- Remove prompt-like comments if they are not required by the assignment.

Optional polish:

- Make the random demo deterministic.

- Avoid assert for validation if main() is meant to be reliable under -O.
- Add type hints.
- Clean up redundant comments.

ASSISTANT OUTPUT

The code review output is the complete review contained in this file.

Concise conclusion:

All 17 supplied tests pass, and the core implementation is correct for its documented ordinary use. There are no blocking defects. The main improvements are to clarify boolean-input semantics, add a few behavioral tests despite already-strong execution coverage, make the randomized demo reproducible if desired, and remove assignment-style comments for a cleaner final submission.